

EQUIPO 03 · UNAM 2026

26 · 05 · 2026

Presentación final

PIPELINE DE DATOS · AWS · BRONZE · SILVER · GOLD

Steam Hidden Gem Score

Un pipeline en AWS que identifica videojuegos de alta calidad con poca visibilidad dentro del catálogo público de Steam.

EQUIPO 3 – INTEGRANTES

De León Vargas Efra Nathaniel

Nava Santiago Erick

Gómez Guzmán Anikey Andrea

Quiroz Flores Héctor Yoram

Toxtega Muñoz Alair

Proyecto Steam HGS

Stack AWS · Python · PostgreSQL

Cohorte UNAM 2026

CONTEXTO DEL PROYECTO

Industria de Videojuegos

— Steam

— 01 / PROBLEMA

Descubrimiento difícil en un catálogo masivo

Steam cuenta con más de 50,000 juegos publicados. Los jugadores no tienen herramientas para descubrir juegos de calidad con poca visibilidad.

50K+ JUEGOS PUBLICADOS EN STEAM

— 02 / SOLUCIÓN

Pipeline de datos en AWS con score propio

Construimos un pipeline de datos en AWS que extrae información pública de Steam, la procesa en capas Bronze-Silver-Gold y calcula un Hidden Gem Score (HGS) para identificar joyas ocultas.

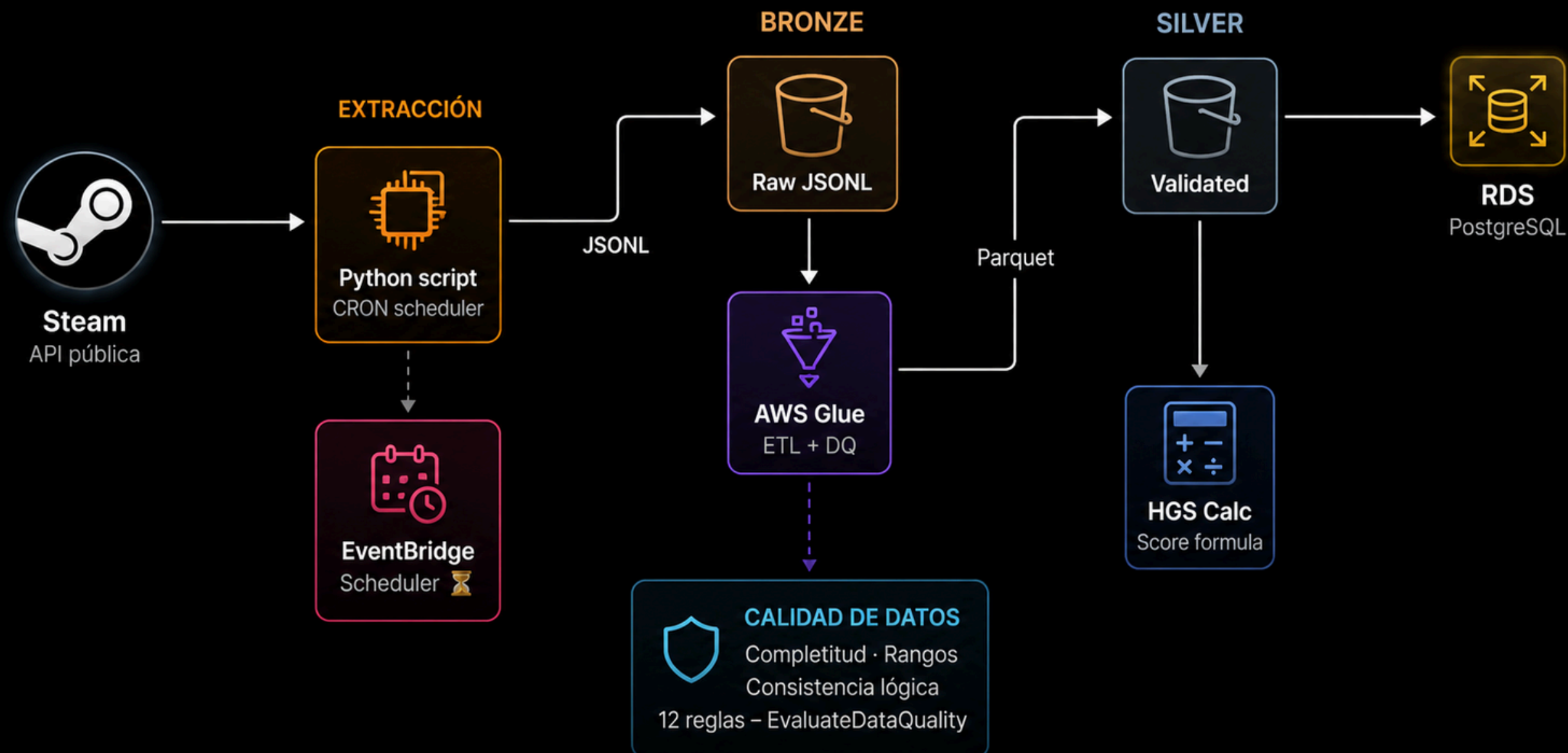
BRONZE · RAW

SILVER · VALIDATED

GOLD · HGS

Arquitectura del Pipeline

BRONZE SILVER GOLD



PROCESOS DEL PIPELINE

Cuatro etapas, un solo flujo

De la Steam Web API a la tabla hidden_gems en RDS

01

EXTRACCIÓN

Steam Web API

Python script consume Steam Web API. Cron diario con EventBridge Scheduler.

EC2 · Python
EventBridge Scheduler

02

BRONZE

Datos crudos en S3

Datos crudos JSONL almacenados en S3 con partición por fecha.

Amazon S3
Formato JSONL

03

ETL SILVER

Glue + Data Quality

AWS Glue aplica 12 reglas de calidad, limpia y escribe Parquet en Silver Layer.

AWS Glue
EvaluateDataQuality

04

GOLD

Cálculo del HGS

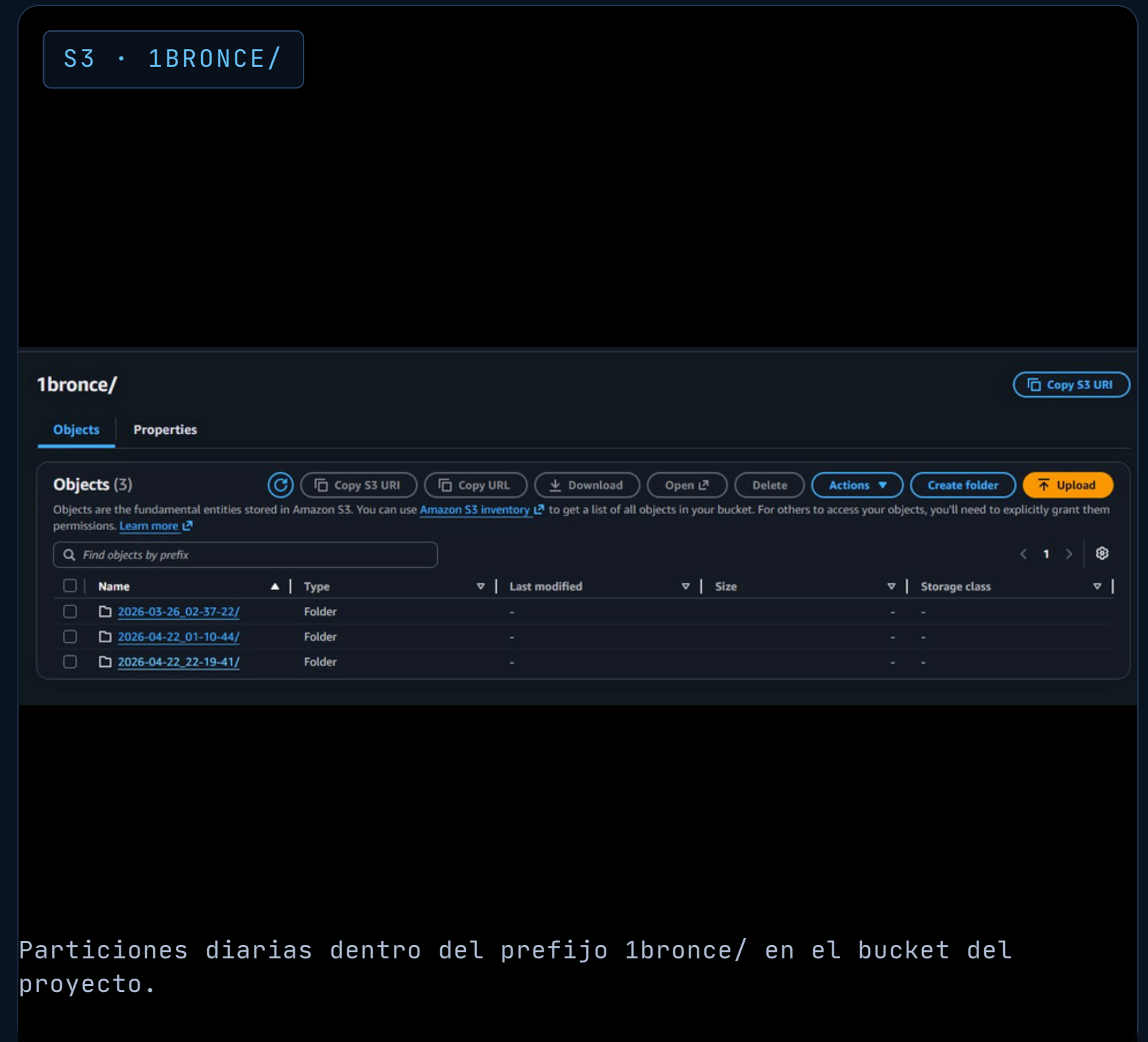
Script Python calcula HGS y carga resultados en RDS PostgreSQL con upsert.

RDS PostgreSQL
Upsert diario

- INGESTA · BRONZE LAYER

Steam Web API → Amazon S3

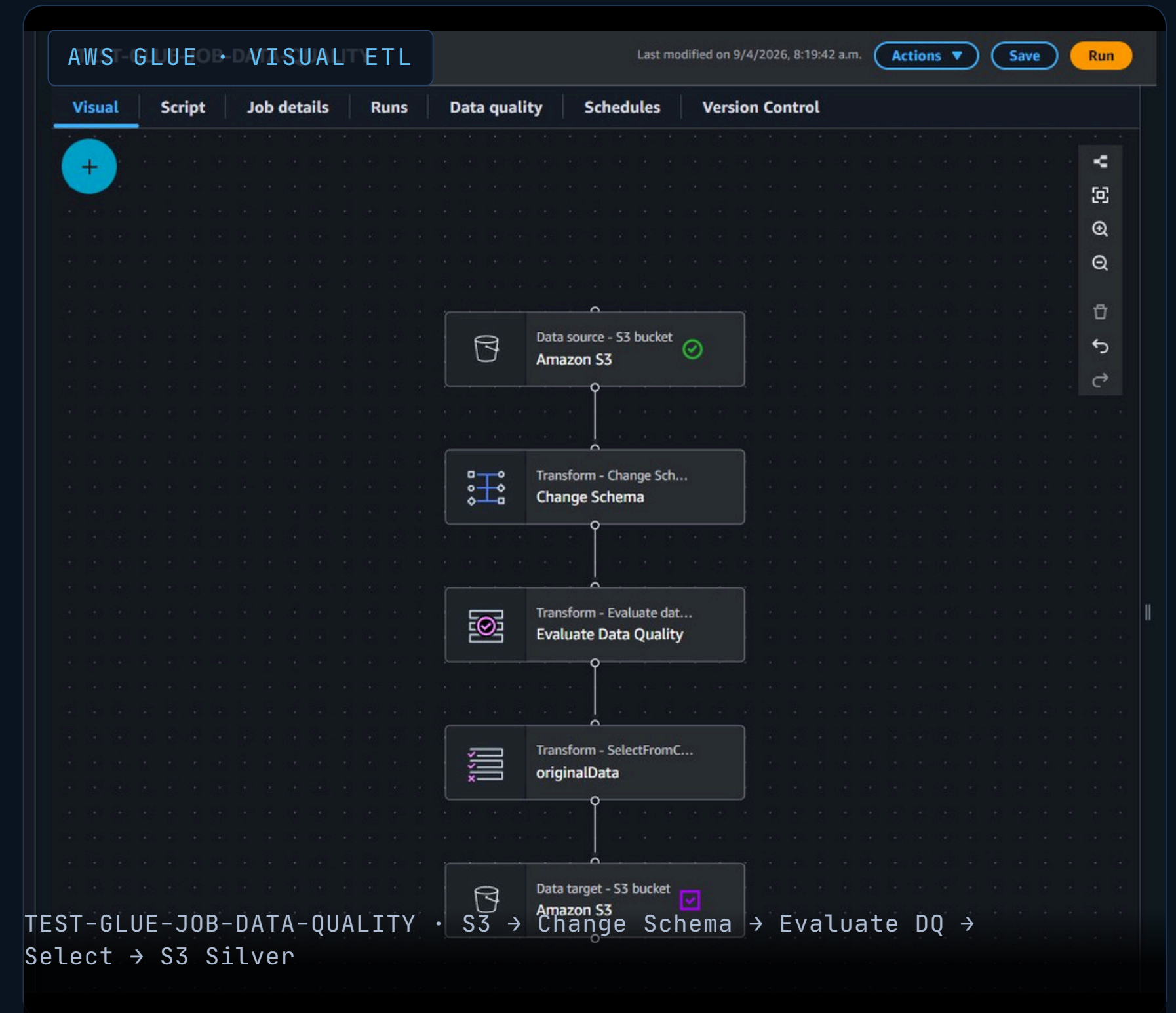
- **01** Script Python consume **GetAppList** y **GetAppDetails** de la Steam Web API pública.
- **02** Ejecutado diariamente mediante **CRON** en **EC2**, orquestado por **AWS EventBridge Scheduler**.
- **03** Los datos se almacenan como **JSONL** en **Amazon S3** (Bronze Layer), con partición por fecha de ejecución.
- **04** Campos capturados: appid, nombre, generos, precio, puntuacion, total_resenas, resenas_positivas.



— BRONZE → SILVER

AWS Glue ETL + Data Quality

- **01 AWS Glue** lee el JSONL desde S3 (Bronze) y ejecuta el job de ETL.
- **02 Limpieza:** eliminación de duplicados, normalización de precios a USD, estandarización de géneros.
- **03 Validación:** 12 reglas EvaluateDataQuality (completitud, rango de precio, reseñas positivas $\leq$ total, etc.).
- **04 Salida:** datos validados escritos en formato **Parquet** en la Silver Layer de S3, listos para el cálculo del HGS.



VALIDACIÓN AUTOMÁTICA

12 Reglas EvaluateDataQuality

COMPLETITUD

Campos obligatorios

appid, nombre, generos, precio y puntuacion no nulos.

RANGO

Valores dentro de límites

Precio entre 0 y 200 USD, puntuación entre 0 y 1, y $\text{resenas_positivas} \leq \text{total_resenas}$.

UNICIDAD

Sin duplicados

appid único por registro, sin duplicados dentro del lote procesado.

FORMATO

Tipos y estructura

appid entero positivo, generos en lista por comas, nombre no vacío.

CUARENTENA

Registros rechazados

Los registros que no cumplen alguna regla van a una **tabla de cuarentena** en S3 para auditoría posterior, sin contaminar la Silver Layer.

- SILVER → GOLD

Hidden Gem Score → RDS

- **01** Script Python lee los Parquet de Silver desde S3 y calcula el **Hidden Gem Score (HGS)**.
- **02** Resultados cargados en **RDS PostgreSQL** con upsert: tabla hidden_gems (por juego) y genre_summary (por género).

FÓRMULA HGS

$$\text{HGS} = 0.50 \cdot \text{calidad} + 0.30 \cdot \text{obscuridad} + 0.20 \cdot \text{precio_accesible}$$
$$\text{Calidad} = \text{resenas_positivas} / \text{total_resenas}$$
$$\text{Obscuridad} = 1 - (\text{total_resenas} / \text{max})$$
$$\text{Precio} = 1 - (\text{precio} / 60)$$

TIER S

$$\text{HGS} \geq 0.85$$

TIER A

$$\text{HGS} \geq 0.70$$

TIER B

resto

```
(RDS · PSQL) ubuntu@ip-172-31-47-107:~$
lmode=verify-full sslrootcert=./gl
Password for user dbadmin:
psql (18.3 (Ubuntu 18.3-1))
SSL connection (protocol: TLSv1.3,
Type "help" for help.
```

```
steam_gold=> \dt
```

List of tables

Schema	Name	Type
public	genre_summary	table
public	hidden_gems	table

(2 rows)

Tablas hidden_gems y genre_summary creadas y pobladas por gold_layer.py.

LA GOLD LAYER RESUELVE EL PROBLEMA

Juegos de calidad con baja visibilidad, identificados y clasificados por HGS

- La tabla `hidden_gems` en RDS contiene el HGS de cada juego con su tier (S, A, B), permitiendo ranking directo.
- La tabla `genre_summary` permite identificar qué géneros concentran más joyas ocultas por HGS promedio.
- El pipeline es re-ejecutable diariamente con upsert: los datos siempre reflejan el estado más reciente de Steam.

SERVICIOS AWS UTILIZADOS

Amazon S3

AWS Glue

EventBridge Scheduler

Amazon EC2

RDS PostgreSQL

S3 · 2silver/

unam-2026-ingenieriadedatos-equipo3-660864588540-us-east-2-an > 2silver/

<

2silver/

Objects | Properties

Objects (4)

Refresh

Copy S3 URI

Copy URL

Download

Open L

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified
☐	run-1775384294686-part-block-0-r-00000-snappy.parquet	parquet	April 5, 2026, 04:18:16 (UTC-06:00)
☐	run-1775744448573-part-block-0-r-00000-snappy.parquet	parquet	April 9, 2026, 08:20:54 (UTC-06:00)
☐	run-1776986126189-part-block-0-r-00000-snappy.parquet	parquet	April 23, 2026, 17:15:31 (UTC-06:00)
☐	run-1779503747099-part-block-0-r-00000-snappy.parquet	parquet	May 22, 2026, 20:35:51 (UTC-06:00)

Archivos .snappy.parquet generados por cada ejecución exitosa del Glue Job.